Inventory Image Tagging via Handcrafted Features and a Small CNN

Manas Selukar Akhil Sharma Anant Singh Tanish Rao Rohan Chaudhary Puneet Takhar
Student project team

***Abstract*—We compare eight classical configurations—logistic regression, k-NN, random forest and RBF-SVM on flattened pixels and HOG—with a small convolutional neural network on Fashion-MNIST. The CNN achieved the highest held-out performance (accuracy 0.9274; macro-F1 0.9269), followed by the pixel RBF-SVM (0.8953; 0.8951). The HOG parameter study selected 6,084-dimensional features using validation data only. The official test set was evaluated once after model selection. The comparison has important limitations: the HOG SVM used a runtime-gated subset of 11,770 training images, the pixel logistic-regression fit emitted a convergence warning at 500 iterations, and the single-pass inference timings are preliminary.**

***Index Terms*—Fashion-MNIST, image classification, HOG, convolutional neural network, support vector machine, benchmarking**

I. Introduction and Problem Statement

This project is Phase 1 (machine-learning evaluation only) of an inventory image tagging system. The question is whether a small convolutional neural network (CNN), which learns spatial features, recognises similar clothing classes better than classical models on fixed representations. The classical models are logistic regression, k-NN, random forest and an RBF-kernel SVM; the fixed representations are flattened pixels and histogram-of-oriented-gradients (HOG) features. The outcome is open: we report what we measure. The final comparison covers nine models (four classical models on two representations, plus one CNN) on Fashion-MNIST.

II. Literature Review and Related Work

A. Fashion-MNIST

Xiao, Rasul and Vollgraf introduce Fashion-MNIST, a set of 70,000 grayscale 28×28 images of fashion products in 10 categories, with 7,000 images per category and an official split of 60,000 training and 10,000 test images [1]. They present it as a direct drop-in replacement for MNIST: it shares the image size, data format and split structure, so existing MNIST code needs essentially only a different download location. Their motivation is that MNIST is popular because it is small, simply encoded and supported by common libraries, but it is an easy task; Fashion-MNIST aims to keep those advantages while being more challenging [1].

The images are front-view thumbnails of 70,000 unique products from an online fashion shop, converted to 28×28 grayscale by a seven-step pipeline. Labels are product silhouette codes assigned manually by in-house experts and reviewed by a separate team. The files use the MNIST format and are stored sorted by label, so shuffling is left to the user [1]; our train/validation split is therefore stratified and seeded. The benchmark in [1] covers scikit-learn classifiers and contains no convolutional network.

B. Learned versus Hand-Designed Features

LeCun et al. review methods for handwritten character recognition and compare them on a standard handwritten digit task [2]. Their abstract argues that multilayer networks trained by back-propagation can learn complex decision surfaces on high-dimensional patterns with minimal preprocessing, and reports that convolutional networks, which are designed for the variability of 2D shapes, outperform the other techniques compared in that study [2]. This motivates the learned-feature side of our comparison.

C. Relation to This Work

We compare a small CNN, which learns spatial features, against four classical models (logistic regression, k-NN, random forest, RBF-SVM) on two fixed representations: flattened pixels and HOG features. Neither cited paper ran this experiment: [1] does not evaluate HOG or a CNN, and [2] studies handwritten digits and documents rather than clothing images. We therefore draw no conclusion from their reported results; our findings are the measurements in Section IX.

III. Dataset Description and EDA

A. Dataset

We use Fashion-MNIST [1]: 28×28 grayscale images of clothing in 10 classes, with an official split of 60,000 training and 10,000 test images. Validation data are a stratified 10% of the official training images (6,000 images, 600 per class, seed 42), leaving 54,000 images for training. The 10,000 test images are not used for any statistic in this section; only the loading checks and the report-only duplicate check read them. Loading checks confirmed uint8 images with pixel values from 0 to 255, int64 labels from 0 to 9, and class names aligned with torchvision. The official training set is exactly balanced, with 6,000 images per class (Fig. 1), and the test set has 1,000 per class.

B. Data quality

We hashed the raw bytes of every image and found no exact duplicates within the training set, within the test set, between training and test, or between our 54,000-image training split and the 6,000-image validation split. This check finds only byte-identical images; near-duplicates were not examined.

C. Pixel statistics

On the 54,000-image training split, the mean pixel value is 0.2862 and the standard deviation is 0.3532 (scale 0–1). Exactly 50.21% of all pixels are zero, which reflects the black background. Mean intensity differs by class, from 0.137 for Sandal and 0.168 for Sneaker to 0.377 for Pullover and 0.385 for Coat. Fig. 2 shows the mean image of each class.

D. Structure of the classes

Class means. The closest pairs of class mean images (Euclidean distance on pixels scaled to 0–1; all pairs are in Fig. 5) are Pullover–Coat (2.473), Pullover–Shirt (2.677), Coat–Shirt (2.990), T-shirt/top–Shirt (3.685) and Sandal–Sneaker (3.823). These distances are descriptive; we do not claim they predict classifier errors, and they are compared with the real confusion matrices later.

PCA. PCA fitted on the training split only gives 71.98% of the variance in the first 10 components and 86.25% in the first 50. The first two components explain 28.99% and 17.77% (46.76% together). In the two-dimensional projection of 5,000 stratified training images (Fig. 4), Shirt, Pullover and Coat overlap heavily, T-shirt/top overlaps with Shirt and Dress, Trouser forms a compact cluster, and Sandal and Sneaker overlap strongly. PCA is linear and unsupervised and the plot shows less than half of the variance, so overlap here does not show that the classes are inseparable.

Visual inspection. In a panel of eight random training images per class (Fig. 3), Shirt images vary widely in pattern and sleeve length. By eye, some plain Shirt images resemble T-shirt/top and some plain long-sleeved Coat images resemble Pullover. This is an impression from 32 images, not a measured rate.

E. Sanity baselines

Two floor references on the validation set: always predicting the majority class gives accuracy 0.100 and macro-F1 0.0182, as expected for 600 of 6,000 validation images per class; nearest class mean gives accuracy 0.692 and macro-F1 0.6882. Both are floors and are not part of the nine-model comparison.

F. Limits of 28×28 grayscale

Each image has only 784 grey values. We hypothesise that this removes fine detail such as buttons, collars and fabric texture, which may make the upper-body classes look alike, and that missing colour removes a further cue. We also hypothesise that images of this style may not match real inventory photos taken at varied size, angle and background. These statements were not tested in this phase.

IV. Methodology and Preprocessing

We use the official split only. The 60,000 official training images are divided into a stratified 54,000-image training split and a 6,000-image validation split with seed 42; the 10,000 official test images are used for evaluation only once, at the final test gate (Section VIII); before the gate they are read only for loading and duplicate checks, and no test statistic

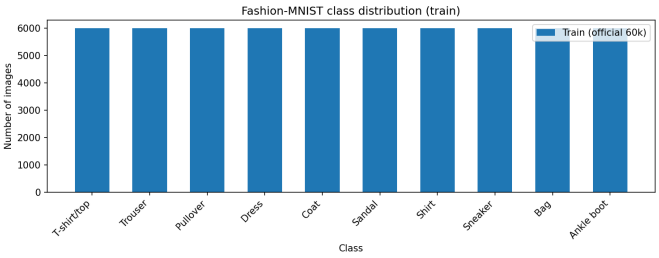


Fig. 1. Number of images per class in the official training set (60,000 images).

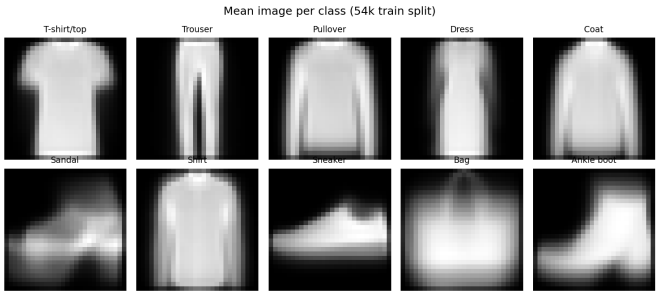


Fig. 2. Mean image of each class on the 54,000-image training split.

enters any fitting or selection step. Nothing (scalers, mean and standard deviation, or any other fitted preprocessing) is fitted on validation or test data. Pixel features are scaled to $[0, 1]$. Logistic regression and the SVM use a standardisation step inside the pipeline; k-NN and random forest use no scaler; the CNN uses the mean and standard deviation of the training split. Final models are trained on the 54,000-image training split only, with no refit on training plus validation.

V. Handcrafted Feature Engineering

We use two fixed representations. *Flattened pixels* are the 28×28 image reshaped row by row into a 784-dimensional vector with values in $[0, 1]$. They keep the absolute grey level at every location but treat each pixel as an independent column: logistic regression, k-NN with Euclidean distance, the RBF-SVM and the random forest give the same results if the columns of all images are permuted in the same way, so these models cannot use the fact that two pixels are neighbours.

Histogram of oriented gradients (HOG) [3] describes local edge structure. We use the implementation in scikit-image [4]: image gradients are computed per pixel; each cell of 4×4 pixels collects a histogram of 9 unsigned orientation bins (0–180°) weighted by gradient magnitude; overlapping blocks of 2×2 cells, moved one cell at a time, are normalised with L2-Hys (L2 normalisation, clipping at 0.2, renormalisation) and concatenated. For a 28×28 image, the selected 2×2-pixel cells give 14 cells per axis; with overlapping 2×2-cell blocks there are 13 blocks per axis. The resulting feature vector has length

$$13 \cdot 13 \cdot (2 \cdot 2) \cdot 9 = 6084. \quad (1)$$

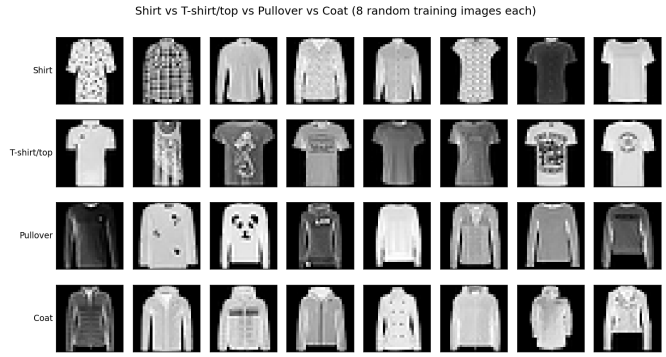


Fig. 3. Eight random training images each of Shirt, T-shirt/top, Pullover and Coat.

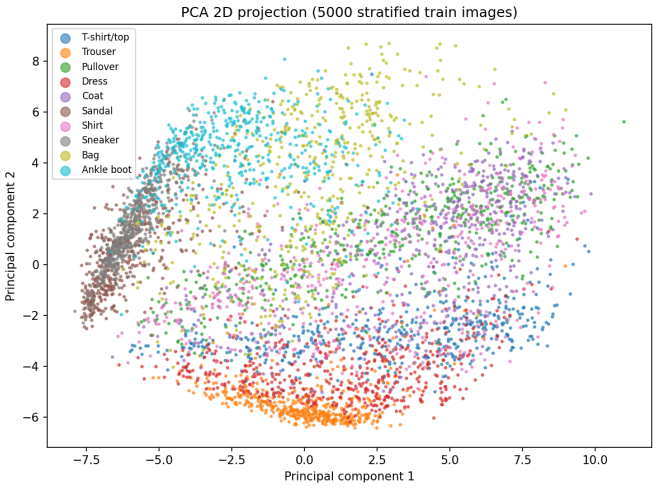


Fig. 4. PCA projection onto the first two components of 5,000 stratified training images, coloured by class. PCA was fitted on the training split only.

which we verified against the library output. Block normalisation makes the descriptor less sensitive to local contrast, but HOG discards absolute grey level.

a) HOG parameter study.: Before any model comparison, the HOG parameters could change once, based on validation data only. Four configurations (the baseline with 4×4-pixel cells, 7×7-pixel cells, 2×2-pixel cells, and 6 orientations) were compared with logistic regression trained on a stratified 10,000-image subset of the training split and scored on the validation split. The decision rule was fixed in advance: switch only if the validation macro-F1 exceeds the baseline by more than 0.010; otherwise keep the baseline. The study uses one model, one subset and one seed, so it is evidence for a choice and not proof of the best descriptor. Outcome: the 2×2-pixel-cell configuration was selected. Its validation macro-F1 was 0.868348, compared with 0.836344 for the 4×4 baseline, an increase of 0.032004, exceeding the pre-set 0.010 threshold. The selected HOG vector has 6,084 features. The comparison used a stratified 10,000-image training subset and validation data only.

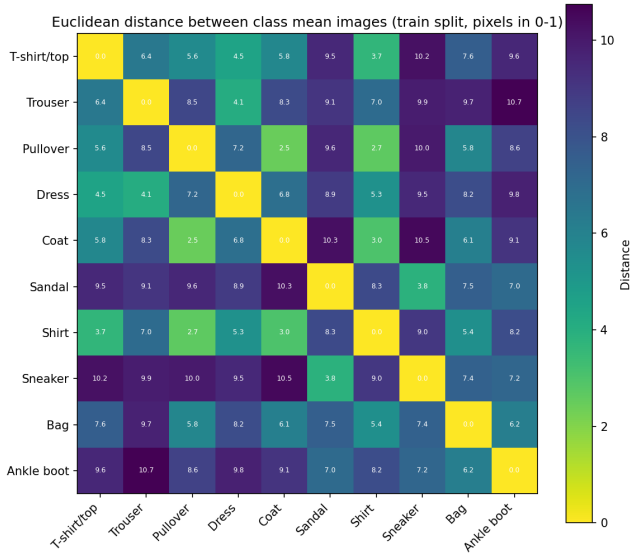


Fig. 5. Euclidean distance between class mean images on the training split (pixels scaled to 0–1).

TABLE I
DATA SPLITS USED IN ALL EXPERIMENTS.

Split	Images	Use
Train	54,000	fitting
Validation	6,000	model selection
Test	10,000	final evaluation, once

b) Leakage.: HOG and flattening are fixed functions applied to each image independently; nothing is fitted, so computing them on all splits creates no leakage. Features are cached on disk and keyed by their kind, split, image count and HOG parameters. The only fitted preprocessing, the standardisation used by logistic regression and the SVM, sits inside the model pipeline and is fitted on the training split.

VI. Classical Machine Learning Models

We compare four classifiers on two representations (flattened pixels, 784 features; HOG, 6084 features), giving eight experiments. All models are trained on the 54,000-image training split; hyperparameters are chosen by validation macro-F1 on the 6,000-image validation split.

A. Logistic Regression

For $K = 10$ classes, the multinomial model gives

$$p(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}, \tag{2}$$

trained by minimising the L2-regularised cross-entropy

$$\min_{W,\mathbf{b}} \ \tfrac{1}{2}\|W\|_F^2 + C \sum_{i=1}^{N} -\log p(y_i \mid \mathbf{x}_i). \tag{3}$$

Features are standardised using training-split statistics only.

TABLE II
HYPERPARAMETER GRIDS (VALIDATION MACRO-F1 SELECTS THE VALUE).

Model	Grid
Logistic regression	$C \in \{0.1, 1.0\}$
k-NN	$k \in \{3, 5, 7, 11\}$
Random forest	minimum leaf size $\in \{1, 3\}$
RBF-SVM	$C \in \{1, 10\}$

B. k-Nearest Neighbours

Following Cover and Hart [5], a query image is assigned the majority class among its k nearest training images under Euclidean distance $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2$. Small k gives low bias and high variance. Training stores the data; the cost is paid at prediction time. Features are not rescaled.

C. Random Forest

A random forest [6] combines trees: each of 200 trees is grown on a bootstrap sample, and each split considers a random subset of features. The forest predicts by averaging tree class probabilities. Splits are threshold tests on single features, so the model is invariant to monotone feature rescaling.

D. RBF Support Vector Machine

The support vector machine [7] uses the kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ and the soft-margin problem is

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t. } y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0. \tag{4}$$

Multiclass prediction uses one-vs-one voting (45 binary classifiers for 10 classes). Hyperparameters were tuned on a stratified 10,000-image subset (see limitations).

E. Hyperparameter grids and runtime gate

The grids searched on the validation split are in Table II; all other settings are library defaults except those stated above (random forest: 200 trees; SVM: RBF kernel with $\gamma =$ "scale"; logistic regression: at most 500 iterations). The selected value is the one with the highest validation macro-F1, with validation accuracy breaking ties. Before every final fit the fit time is estimated; a model whose estimate exceeds 30 minutes would be fitted on a disclosed stratified subset, and the number of training images used is always reported. In the completed run, seven classical configurations used all 54,000 training images. The HOG RBF-SVM was runtime-gated to a stratified subset of 11,770 images because its estimated full fit exceeded the 30-minute limit. Final fit times ranged from 0.02 seconds for pixel k-NN to 393.61 seconds for HOG logistic regression. The pixel logistic-regression fit reached the configured 500-iteration limit with a convergence warning; its result is reported as run, not as proof of convergence.

VII. CNN Architecture and Training

The CNN has three convolution blocks followed by a two-layer classifier (Table III). Inputs are 28×28 grayscale images, scaled to $[0, 1]$ and standardised with the mean and standard deviation of the 54,000-image training split only. The network outputs ten raw logits; the softmax is applied inside the loss and when probabilities are reported.

TABLE III
CNN LAYERS AND TRAINABLE PARAMETERS (THE TOTAL IS VERIFIED BY CODE).

Layer	Output shape	Params	Calculation
Conv 3×3, 1→32, ReLU, MaxPool 2	32×14×14	320	$32(1 \cdot 9 + 1)$
Conv 3×3, 32→64, ReLU, MaxPool 2	64×7×7	18,496	$64(32 \cdot 9 + 1)$
Conv 3×3, 64→128, ReLU, MaxPool 2	128×3×3	73,856	$128(64 \cdot 9 + 1)$
Flatten, Dropout, Linear, ReLU	128	147,584	$1152 \cdot 128 + 128$
Dropout, Linear	10	1,290	$128 \cdot 10 + 10$
Total		241,546	

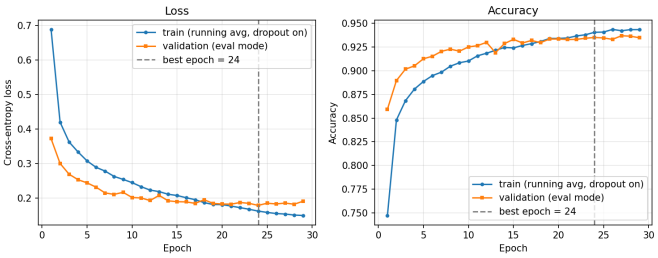


Fig. 6. CNN training and validation learning curves; the dashed line marks the epoch whose weights were kept.

Each convolution uses padding 1, so only pooling changes the spatial size; $7 \to 3$ uses floor division, so the last row and column of the 7×7 map are ignored. Dropout [8] ($p = 0.5$) is applied only in the classifier, which holds most of the parameters.

a) Training.: We minimise the cross-entropy loss with Adam [9] (learning rate 10^{-3}, batch size 128, at most 30 epochs). Mini-batches are shuffled with a seeded generator. After every epoch the validation loss and accuracy are computed in evaluation mode (dropout off). Training stops when the validation loss has not improved for 5 consecutive epochs, and the weights of the best epoch are restored; the test images play no role in this procedure. Training loss and accuracy in Fig. 6 are running averages over the mini-batches of each epoch with dropout active, so they are pessimistic compared with an evaluation-mode pass. The learning rate, batch size and dropout were not tuned. All results are from a single run with seed 42; run-to-run variation is not measured. The recorded run lasted 29 epochs and restored the best checkpoint from epoch 24 (validation accuracy 0.9352, validation loss 0.1783). Training took 49.01 seconds on a Tesla T4 GPU. The final test accuracy was 0.9274 and macro-F1 was 0.9269.

VIII. Experimental Protocol and Evaluation Metrics

Model selection uses the highest validation macro-F1, with validation accuracy breaking ties; the SVM is tuned on a stratified 10,000-image subset of the training split. Before every final classical fit the fit time is estimated, and if it

exceeds the limit the model is fitted on a disclosed stratified subset; the number of training images used is always reported. After selection the chosen configurations are written to a freeze file and all nine models are evaluated on the test images exactly once.

a) Metrics.: Accuracy is the fraction of correctly classified images. For class c, $F1_c = 2TP_c/(2TP_c + FP_c + FN_c)$, and macro-F1 is the unweighted mean of the ten $F1_c$ values, so every class counts equally. The test set has exactly 1,000 images per class, so macro-averaged recall equals accuracy; macro-F1 differs because it also depends on precision. Confusion matrices have the true class in rows and the predicted class in columns.

b) Uncertainty and paired comparison.: We report 95% percentile bootstrap confidence intervals [10]: the 10,000 test images are resampled with replacement 1000 times (seed 42). These intervals capture the sampling noise of this test set for one trained model; they do not include the randomness of training. To compare the CNN with each classical model on the same images we use McNemar's test [11] on the paired correctness of the two models (exact binomial test when there are fewer than 25 discordant pairs, otherwise a chi-square statistic with continuity correction), with a Bonferroni correction for the eight comparisons. This test also ignores training variance [12].

c) Inference cost.: Timing uses the same 10,000 test images in a fixed order, and excludes model loading, training and plotting. For the classical models we report "predict only" (precomputed features) and "end-to-end" (feature extraction from the raw images, then prediction) separately, as one timed pass after an untimed warm-up call in the saved run. These single-repeat values are preliminary, not stable latency estimates. For the CNN the inputs are pre-normalised tensors, batch size 256, with three warm-up batches; the recorded table contains a single timing per device. CNN tensor timing excludes image preprocessing and is not directly comparable to classical end-to-end timing. The notebook records the library versions and the hardware it ran on (`results/environment.json`); implementations are scikit-learn [13], PyTorch [14] and scikit-image [4].

IX. Results and Comparative Analysis

X. Confusion Matrices and Error Analysis

The CNN made 726 errors out of 10,000 test images (error rate 7.26

XI. Discussion, Limitations and Future Improvements

The CNN outperformed all eight classical configurations on the held-out test set, reaching 92.74% accuracy and 0.9269 macro-F1. The strongest classical result was the pixel RBF-SVM at 89.53% accuracy and 0.8951 macro-F1. The CNN also had the lowest observed error count. The paired McNemar comparisons were significant after correcting for eight comparisons. Inference timings suggest a trade-off: the CNN's recorded GPU tensor latency was 0.0116 ms/image, but that excludes preprocessing; classical end-to-end timings include feature extraction and should not be compared to that figure directly.

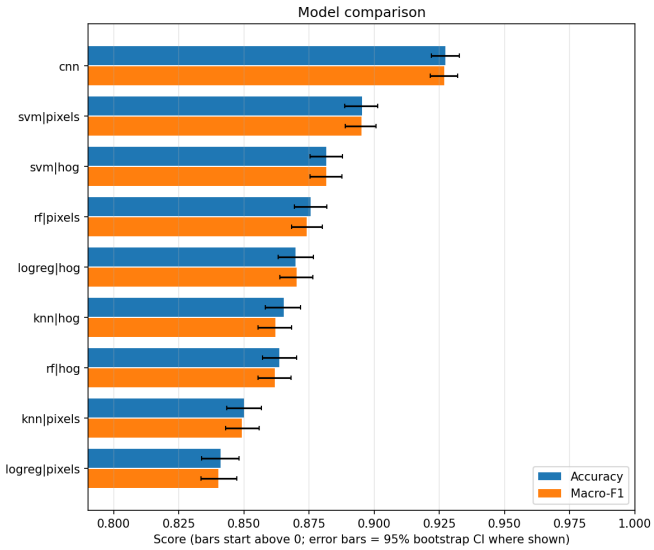


Fig. 7. Accuracy and macro-F1 of the nine models on the test set with 95% bootstrap confidence intervals.

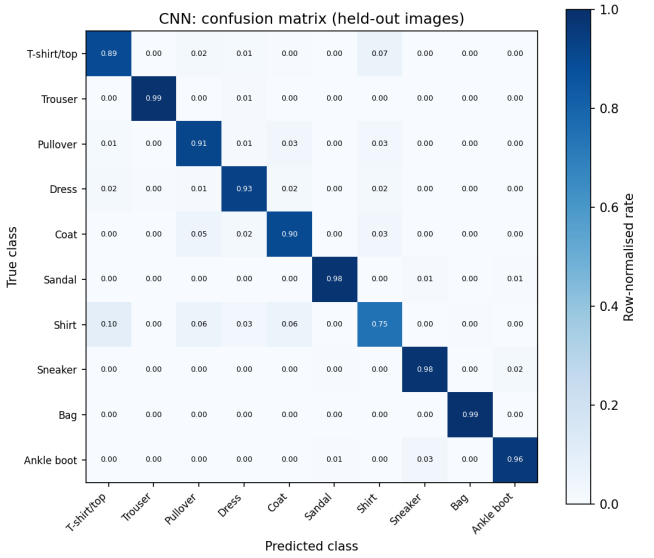


Fig. 8. Row-normalised confusion matrix of the CNN on the test set (rows: true class).

Several limitations qualify these findings. The evaluation covers one benchmark dataset, one split and one random seed; it does not establish performance on real inventory photographs or quantify training-seed variability. The CNN hyperparameters were not tuned, while the classical models used small grids. The pixel logistic-regression fit reached the 500-iteration cap with a convergence warning. The HOG RBF-SVM used only 11,770 of 54,000 training images because its estimated full fit exceeded the 30-minute runtime gate, so

TABLE IV

HELD-OUT TEST PERFORMANCE. PARENTHESES SHOW 95% PERCENTILE BOOTSTRAP CONFIDENCE INTERVALS FROM 1,000 RESAMPLES. THE OFFICIAL TEST GATE WAS RUN ONCE.

Model	Representation	n_{train}	Accuracy (95% CI)	Macro-F1 (95% CI)
Logistic regression	pixels	54,000	0.8411 (0.8338–0.8482)	0.8403 (0.8336–0.8474)
k-NN	pixels	54,000	0.8500 (0.8434–0.8567)	0.8493 (0.8430–0.8559)
Random forest	pixels	54,000	0.8756 (0.8693–0.8818)	0.8741 (0.8682–0.8800)
RBF-SVM	pixels	54,000	0.8953 (0.8887–0.9014)	0.8951 (0.8889–0.9008)
Logistic regression	HOG	54,000	0.8698 (0.8632–0.8766)	0.8702 (0.8637–0.8764)
k-NN	HOG	54,000	0.8653 (0.8583–0.8718)	0.8621 (0.8555–0.8682)
Random forest	HOG	54,000	0.8636 (0.8571–0.8703)	0.8618 (0.8554–0.8682)
RBF-SVM	HOG	11,770	0.8817 (0.8753–0.8879)	0.8816 (0.8755–0.8877)
Small CNN	learned	54,000	0.9274 (0.9219–0.9327)	0.9269 (0.9214–0.9321)

TABLE V

INFERENCE TIMING ON 10,000 TEST IMAGES. CLASSICAL VALUES ARE SINGLE TIMED PASSES AFTER AN UNTIMED WARM-UP; TREAT AS PRELIMINARY. PREDICT-ONLY USES PRECOMPUTED FEATURES, WHILE END-TO-END INCLUDES FEATURE EXTRACTION. CNN CPU/GPU TIMING USES PRE-NORMALISED TENSORS AND IS NOT DIRECTLY COMPARABLE TO CLASSICAL END-TO-END TIMING.

Model	Representation	Predict-only CPU (ms/img)	End-to-end CPU (ms/img)	CNN GPU (ms/img)	Fit time (s)
Logistic regression	pixels	0.008830	0.010633	–	131.48
k-NN	pixels	2.255952	2.120674	–	0.02
Random forest	pixels	0.056275	0.048987	–	125.09
RBF-SVM	pixels	14.804295	14.560057	–	267.66
Logistic regression	HOG	0.056755	3.302105	–	393.61
k-NN	HOG	15.137228	18.482470	–	0.12
Random forest	HOG	0.082839	3.329037	–	348.48
RBF-SVM	HOG	48.426658	51.899920	–	322.48
Small CNN	learned	0.440153	not measured	0.011625	49.01

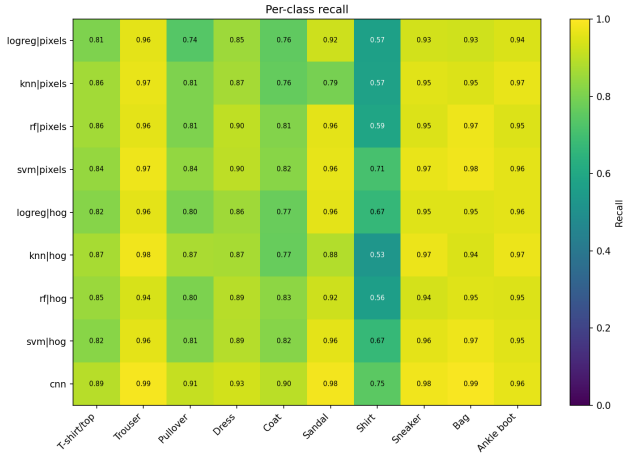


Fig. 9. Per-class recall of the nine models on the test set.

its score is not directly comparable to full-data fits. Inference timings were recorded as single timed passes after warm-up and should be repeated for robust latency estimates. Future work should validate on real inventory images, repeat training across seeds, expand hyperparameter searches within a documented budget, investigate logistic-regression convergence, and benchmark all models with repeated runs under matched preprocessing and hardware conditions.

XII. CONCLUSION

For this Fashion-MNIST benchmark, the small CNN was the strongest evaluated model, with 0.9274 test accuracy and 0.9269 macro-F1. Among classical models, the pixel RBF-SVM performed best (0.8953 accuracy, 0.8951 macro-F1). The selected HOG representation improved validation macro-F1 over the original HOG baseline, but the runtime-gated HOG SVM fit and other protocol limitations should be considered when interpreting the comparison. The results support the CNN for this benchmark, not yet for production inventory tagging without further real-world validation.

ACKNOWLEDGMENT

The repository snapshot did not specify external funding or additional acknowledgments.

REFERENCES

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017. [Online]. Available: https://arxiv.org/abs/1708.07747

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, dOI 10.1109/5.726791.

[3] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2005, pp. 886–893.

[4] S. van der Walt *et al.*, "scikit-image: image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.

[5] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[7] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
[8] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015, arXiv:1412.6980.
[10] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
[11] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[12] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
[13] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, arXiv:1912.01703.